

Day 50 -- Line-by-Line Syntax Explanation

day50_dropout_regularization_comparison.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 8, Day 2 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula, a purple MATH box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green SYSTEM DIAGRAM shows how that math and code fit together as a working system.

Segment 0 -- Imports and Shared Helpers (reused verbatim from Days 39-49)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
np.random.seed(42)
def relu(z): return np.maximum(0, z)
def drelu(z): return (z > 0).astype(z.dtype)
def softmax(z): ...
def one_hot(y, n_classes): ...
def cross_entropy_loss(probs, y_onehot): ...
def he_scale(n_in): return np.sqrt(2.0 / n_in)
def conv2d_forward_mc(X, W, b): ...
def conv2d_backward_mc(dZ, X, W): ...
def maxpool_forward(A, size=2, stride=2): ...
def maxpool_backward(dOut, A, idx_cache, size=2, stride=2): ...
def iterate_minibatches(X, y_onehot, batch_size, rng, shuffle=True): ...
def make_junction_dataset_rgb(...): ...
def dropout_forward(A, keep_prob, training, rng): ...
def dropout_backward(dOut, mask): ...
class AlexNetStyleConvNet: ...
```

None of this is new today -- every function and class here, `dropout_forward/dropout_backward` and `AlexNetStyleConvNet` included, is a character-for-character copy of Day 49's own code, unchanged. This is a deliberate choice, not laziness: today's entire point is that the SAME math, given a fairer training budget, tells a different story -- reusing the code verbatim is what makes that comparison valid rather than confounding a code change with a training-budget change.

Segment 1 -- `train_with_history` (NEW today)

```
def train_with_history(net, X, y, X_test, y_test, epochs, batch_size,
                      eval_every=10, n_classes=4, seed=0):
    y_oh = one_hot(y, n_classes)
    rng = np.random.RandomState(seed)
    step_losses = []
    history_epochs, history_train_acc, history_test_acc = [], [], []
    for epoch in range(epochs):
        for Xb, yb, idx in iterate_minibatches(X, y_oh, batch_size, rng):
            probs = net.forward(Xb, training=True, rng=rng)
            step_losses.append(cross_entropy_loss(probs, yb))
            net.backward(yb)
        if epoch % eval_every == 0 or epoch == epochs - 1:
            history_epochs.append(epoch)
            history_train_acc.append(net.accuracy(X, y))
            history_test_acc.append(net.accuracy(X_test, y_test))
    return dict(step_losses=step_losses, epochs=history_epochs,
               train_acc=history_train_acc, test_acc=history_test_acc)
```

The inner mini-batch loop (`for Xb, yb, idx in iterate_minibatches`) is byte-for-byte the same shape as every prior day's `.train()` method -- what's new is the `if epoch % eval_every == 0 or epoch == epochs - 1:` block after it. The

modulo check evaluates on epoch 0, 10, 20, ... and the `epoch == epochs - 1` guarantees the very last epoch is always included even when it doesn't happen to be a multiple of `eval_every` (150 epochs with `eval_every=10` happens to divide evenly, but the guard makes the function correct regardless of that coincidence). Both accuracy calls go through `net.accuracy`, which internally forces `training=False` -- evaluation never accidentally applies dropout.

SYSTEM DIAGRAM -- one epoch of `train_with_history`: a full mini-batch pass, then a conditional train/test accuracy checkpoint every `eval_every` epochs.

Segment 2 -- Quick Re-Verification

```
rng = np.random.RandomState(0)
A = rng.rand(2000, 50) + 0.5
sum_out = np.zeros_like(A)
for _ in range(2000):
    out, _ = dropout_forward(A, 0.5, training=True, rng=rng)
    sum_out += out
grand_rel_err = abs((sum_out / 2000).sum() - A.sum()) / abs(A.sum())
```

A condensed rerun of Day 49's own pooled statistical check -- same logic (accumulate 2000 independent `dropout_forward` calls on a fixed input, compare the pooled mean's sum against the original input's sum), fewer lines because the per-unit noise analysis and the `training=False/backward-passthrough` checks Day 49 already ran in full aren't repeated here.

Segment 3 -- The Main Comparison

```
epochs, lr, batch_size, eval_every = 150, 0.15, 50, 10

net_reg = AlexNetStyleConvNet(img_size=img_size, lr=lr, keep_prob=0.5, seed=42)
hist_reg = train_with_history(net_reg, X_train, y_train, X_test, y_test,
                             epochs=epochs, batch_size=batch_size,
                             eval_every=eval_every, seed=42)

net_unreg = AlexNetStyleConvNet(img_size=img_size, lr=lr, keep_prob=1.0, seed=42)
hist_unreg = train_with_history(net_unreg, X_train, y_train, X_test, y_test,
                               epochs=epochs, batch_size=batch_size,
                               eval_every=eval_every, seed=42)
```

Both networks are constructed with the identical `seed=42` -- same weight initialization -- and trained with the identical `seed=42` passed to `train_with_history`, so the ONLY difference between `net_reg` and `net_unreg` is `keep_prob`. This is the same single-variable-changed discipline Day 49's own ablation used, just now at a training budget long enough for the comparison to actually mean something.

```
best_idx_unreg = int(np.argmax(hist_unreg["test_acc"]))
best_epoch_unreg = hist_unreg["epochs"][best_idx_unreg]
best_test_unreg = hist_unreg["test_acc"][best_idx_unreg]
```

`np.argmax` on the tracked `test_acc` history finds WHICH recorded evaluation point had the best test accuracy -- `hist_unreg["epochs"][best_idx_unreg]` then looks up which actual epoch number that corresponds to. This is exactly the computation an early-stopping / checkpointing rule performs continuously during training; here it's done once, after the fact, on the full recorded history.

SYSTEM DIAGRAM -- two `AlexNetStyleConvNet` instances, identical seed/data, differing only in `keep_prob`, each producing its own tracked history.

Segment 4 -- The Comparison Plot

```
plt.plot(hist_unreg["epochs"], hist_unreg["train_acc"],
         label="dropout OFF -- train_acc", color="#d9534f", linestyle="--")
plt.plot(hist_unreg["epochs"], hist_unreg["test_acc"],
```

```

        label="dropout OFF -- test_acc", color="#d9534f")
plt.plot(hist_reg["epochs"], hist_reg["train_acc"],
        label="dropout ON -- train_acc", color="#337ab7", linestyle="--")
plt.plot(hist_reg["epochs"], hist_reg["test_acc"],
        label="dropout ON -- test_acc", color="#337ab7")
plt.axvline(best_epoch_unreg, color="#d9534f", alpha=0.3, linewidth=1)
plt.axvline(best_epoch_reg, color="#337ab7", alpha=0.3, linewidth=1)

```

Four lines, one color per configuration (red = dropout OFF, blue = dropout ON) and one linestyle per metric (dashed = train_acc, solid = test_acc) -- so all four curves are readable on one shared axis without a legend entry per line being ambiguous about which network or which metric it belongs to. The two `axvline` calls draw a thin vertical marker at each configuration's own best-test-accuracy epoch, directly on the plot, at the same x-position `best\_epoch\_unreg`/`best\_epoch\_reg` computed in Segment 3.

Segment 5 -- What This Maps To in PyTorch, and main()

```

def main():
    demo_quick_reverification()
    demo_converged_comparison()
    note_on_pytorch()

```

The same ordering principle every earlier day has followed: verify the reused code still behaves correctly FIRST, then run the real comparison, then relate it back to a production framework. `note\_on\_pytorch` is a pure documentation function, printing a PyTorch-equivalent checkpointing loop as a string rather than importing torch -- the same pattern every prior day's own note_on_pytorch()-style function has used.

Quick Syntax Glossary -- New or Reinforced Constructs

``if epoch % eval_every == 0 or epoch == epochs - 1:`` (periodic check with a guaranteed final evaluation)

New today: a modulo condition that fires every `eval_every` epochs, OR'd with an explicit check for the very last epoch so the final state is always recorded even if `epochs-1` isn't a multiple of `eval_every`. Without the ``or`` clause, a run whose epoch count doesn't divide evenly by `eval_every` would silently never report its true final accuracy.

``np.argmax(hist["test_acc"])`` then indexing back into ``hist["epochs"]`` (finding WHEN, not just WHAT)

`argmax` on the accuracy list gives a POSITION in that list, not an epoch number -- the two tracked lists (`epochs`, `test_acc`) stay in parallel throughout `train_with_history` specifically so that position can be looked up in the `epochs` list afterward, turning 'which recorded value was best' into 'which epoch was best.'

Two lines per configuration, one linestyle each (``linestyle="--"`` vs. `solid`)

Reusing the SAME color per network across both its `train_acc` and `test_acc` lines (rather than four unrelated colors) makes the train/test GAP for each network visually obvious as the vertical distance between its own dashed and solid line, which is the actual quantity this lesson cares about.

Reusing an entire class and two functions verbatim across files (no import, full copy-paste)

This series has never imported one day's script from another -- each day's `.py` file is self-contained and runnable on its own, so genuinely unchanged code (`dropout_forward/backward`, `AlexNetStyleConvNet`) gets copied character-for-character rather than imported. The tradeoff is explicit here so it doesn't read as an oversight: self-containment costs some duplication.

End of Day 50 syntax reference. For the concepts these lines implement, see the main Day 50 study guide PDF.